Gain Tuning for PI Controllers Using Machine Learning

Background

In modern control systems, achieving optimal performance under varying operating conditions and disturbances is a critical challenge. Traditional PI controllers often require manual tuning of proportional and integral gains, which can be time-consuming and suboptimal when system dynamics change. Machine Learning (ML) offers a promising approach to adaptively tune controller gains based on real-time feedback and system behaviour.

Objective

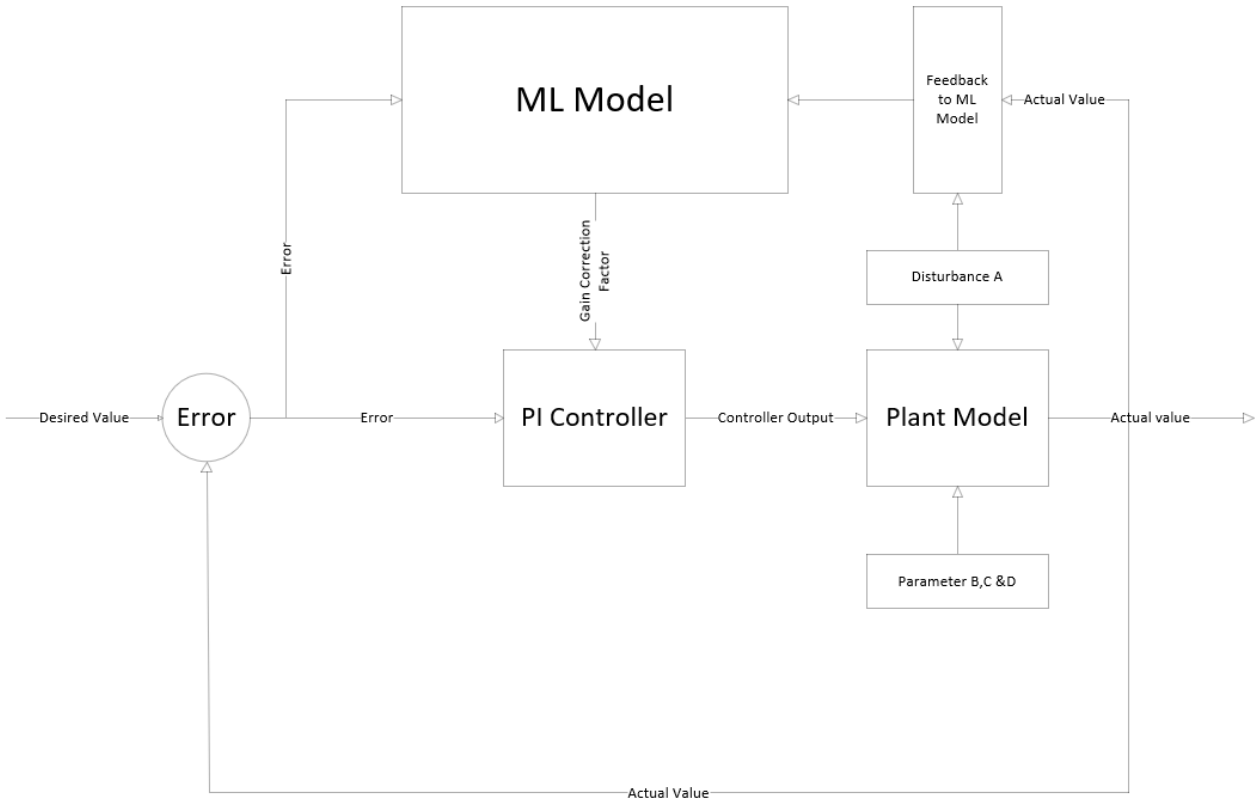
Design and implement an ML-based solution that dynamically adjusts the PI controller gains to minimize error and improve system stability for a given plant model. The gain correction factor is a multiplication on the existing proportional and integral gains provided with the controller. The gain correction factor depends on the error (The difference between desired and actual value) and the imposed disturbance. The ML model should learn from system feedback, requirement parameters and disturbances to provide gain correction factors that enhance controller performance.

System Overview

The control loop consists of:

- **Error Block**: Computes the difference between desired value and actual value.
- **PI Controller**: Generates control output based on error and tuned gains.
- **Plant Model**: Represents the system dynamics influenced by parameters (B, C, D) and disturbance (A).
- **ML Model**: Can use error, Disturbance A, requirement parameters (defined in Requirements figure), plant defining parameters (parameter B, C, &D) and actual value feedback, and outputs gain correction factors for the PI controller. The gain correction factor should only be dependent on the error and the Disturbance (A).

Caterpillar: Confidential Green



Inputs Provided

A. A **plant transfer function** with the operating range of disturbance A.

B. **PI Controller transfer function** and the fixed gains (On top of which the ML based correction should be applied).

C. The requirements parameter range (OS%, US%, T(sec), etc) – mentioned in the Requirements Figure.

D. A disturbance profile (Disturbance A).

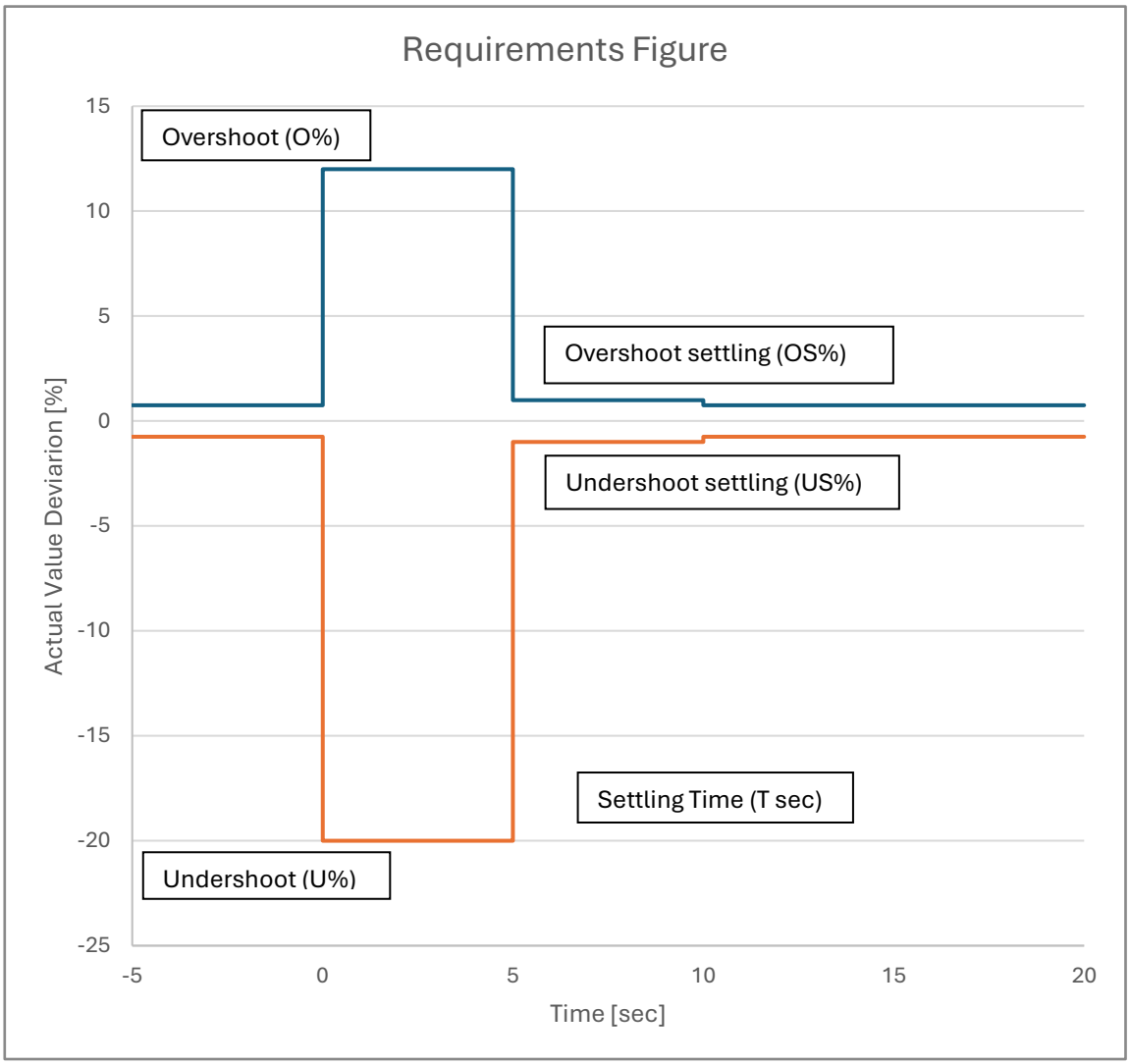
E. Desired setpoint values for the system.

Expected Deliverables

1. **ML Model Design**:
    - Choose any ML approach (e.g., regression, reinforcement learning, neural networks).
    - Train the model to predict gain correction factors based on error, disturbance and system feedback.

2. **Simulation Environment**:
    - Implement the PI controller and plant model in MATLAB.
    - Integrate the ML model into the control loop.

3. **Performance Metrics**:
    - Fulfil the requirements criteria for different disturbance rejection mentioned below
    - Compare performance against a baseline PI controller with fixed gains.



Requirement 1 (Incremental disturbance)

Requirement 2 (Decremental disturbance)

Evaluation Criteria

- Accuracy and adaptability of the ML-based tuning.
- Robustness under varying disturbances and parameter changes.
- Creativity in model selection and integration.
- Visualization of results (plots of error, controller output, and gain adjustments).

Available Data

1. Simulink Model with relevant plant model and disturbance modelled provided as ***ProblemState.mdl***.
2. Sample simulation data for the Simulink model has also been provided as ***Simulink_Data.mat*.**
3. Input value fixed at 900 RPM (no change required)
4. Disturbance can vary with the range $[0,1]$.

For the problem, the model utilizes fixed proportional and integral gains ($K_{p_{fixed}}, K_{i_{fixed}}$) for the closed loop model. Develop an algorithm (*no restrictions*) of to compute the correction factors over the existing gains such that

$$K_{p_{corr}} = f(Disturbance, error)$$

$$K_{i_{corr}} = f(Disturbance, error)$$

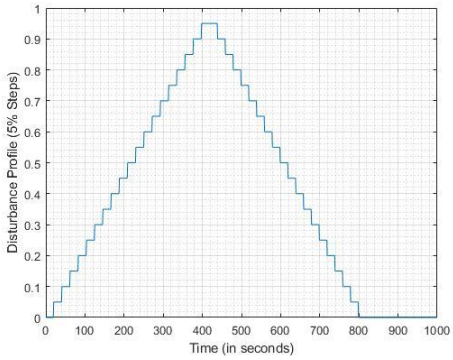
Therefore, the final proportional and integral gains become

$$K_{p_{final}} = K_{p_{fixed}} \times K_{p_{corr}}$$

$$K_{i_{final}} = K_{i_{fixed}} \times K_{i_{corr}}$$

Additional Data can be generated via simulations using following disturbance profiles

A. 5% steps-sheds (increase disturbance by 5% uniformly from 0 to 100% and back to 0%)



Performance metrics:

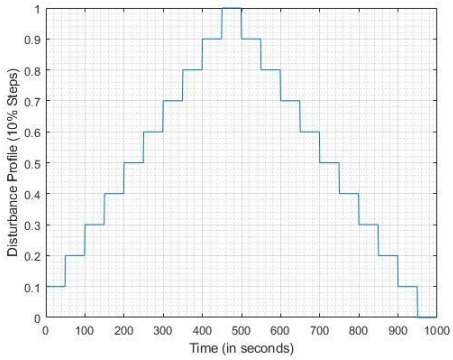
Maximum %Overshoot: ***15%***

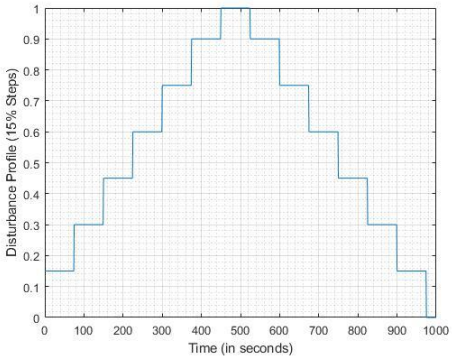
Maximum %Undershoot: ***15%***

Settling Time***: 3 seconds (error<1%)***

Steady State Time: ***8 seconds (error<0.25%)***

B. Repeat the above activity for 10% and 15% step/shed sizes as shown in the figure below





Performance metrics:

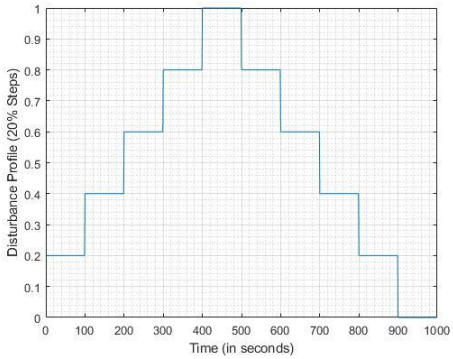
Maximum %Overshoot: ***20%***

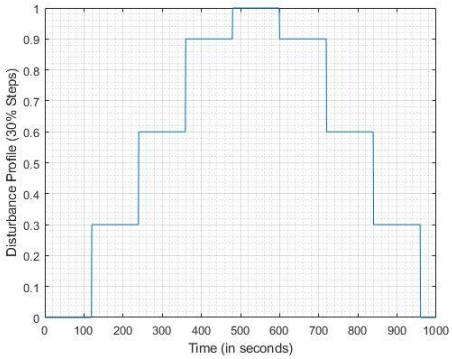
Maximum %Undershoot: ***20%***

Settling Time***: 5 seconds (error<1%)***

Steady State Time: ***10 seconds (error<0.75%)***

C. Repeat the above activity for 20% and 30% step/shed sizes as shown in the figure below





Performance metrics:

Maximum %Overshoot: ***25%***

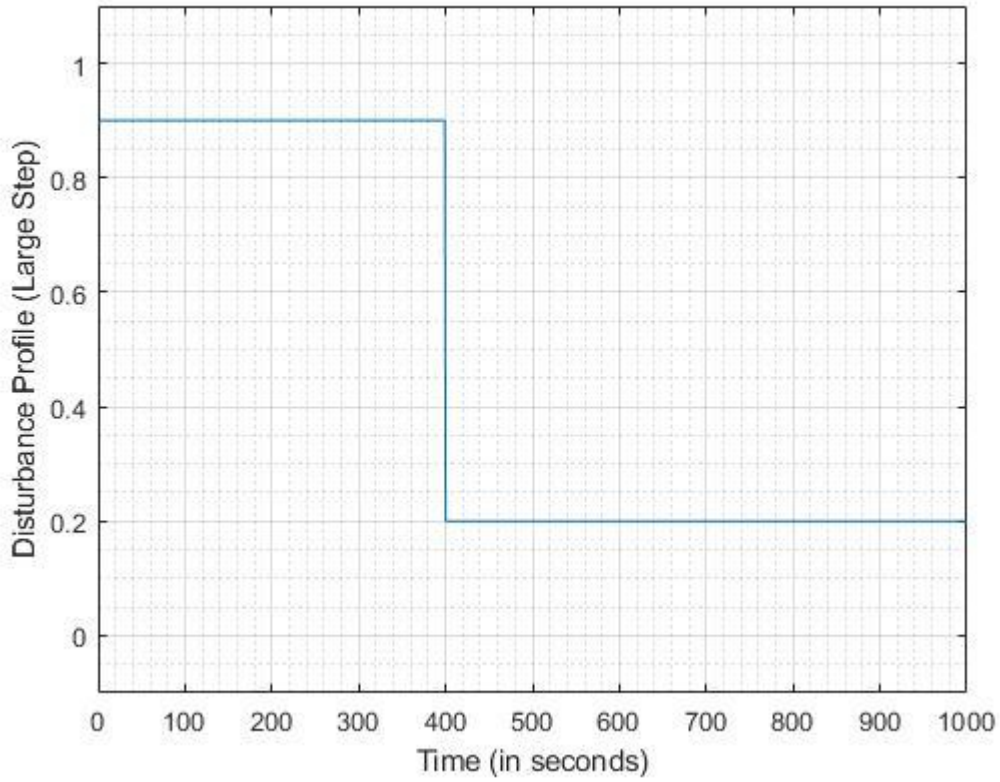
Maximum %Undershoot: ***25%***

Settling Time***: 10 seconds (error<1.75%)***

Steady State Time: ***15 seconds (error<1.25%)***

Find Bandwidth, Gain Margin and Phase Margin for all the above cases

Fine tune your algorithm to optimize gains and hence performance for the following disturbance profile



1. Algorithms would be evaluated based on system response for the above test case
2. Time Complexity/ Space Complexity is not a primary criterion for evaluation